

Material de Apoio: Fundamentos de Pandas na Saúde

1. Importação das Bibliotecas e o Apelido `pd`

O que é o Pandas? O Pandas é considerado o "Excel do Python". É uma biblioteca construída especificamente para analisar e manipular tabelas de dados.

- **De onde vem o nome?** Vem de *Panel Data* (Dados em Painel), um termo da estatística. Não tem relação com o urso!
- **Por que não usar o Excel?** O Pandas não trava. Ele processa milhões de linhas de pacientes em segundos.
- **Para que serve na prática?** Abrir, limpar e cruzar enormes volumes de dados clínicos para preparar o terreno para a Inteligência Artificial.

O apelido `pd`: Para começar a usar, basta "chamar" essa ferramenta no seu código usando `import pandas as pd`. A sigla "pd" é apenas um apelido carinhoso para não precisarmos digitar a palavra inteira toda vez.

2. Series (1D) vs DataFrame (2D) & Criação

Como o Pandas organiza um sistema de saúde?

- **DataFrame (2D):** É a tabela inteira (linhas e colunas). Imagine a ficha completa com os dados de todos os pacientes.
- **Series (1D):** É uma única coluna. Imagine que você arrancou apenas a coluna "idade" para analisá-la isoladamente.

```
In [31]: import pandas as pd
import numpy as np
print("Bibliotecas importadas com sucesso!")

# Criando a tabela exatamente com os dados mostrados nos seus slides
dados = pd.DataFrame({
    "paciente": ["Ana Lima", "João", "Maria", "Carlos", "Fernanda", "Lucas",
    "idade": ["45", "62", "38", "55", "29", "71", "48", "62"], # Como texto para
    "glicemia": [99.4582, 140.1234, np.nan, 115.0, -1.0, 158.0, 105.0, 140.1234]
    "sangue": ["A+", "O+", "B+", "AB+", "O-", "A-", "B-", "O+"],
    "cidade": ["Campina Grande", "João Pessoa", "Recife", "Natal", "Fortaleza",
    "sexo": ["F", "Masc", "Feminino", "M", "F", "M", "Feminino", "Masc"],
    "data_admissao": ["2024-01-15", "2024-02-10", "2024-01-20", "2024-02-10", "2
    "diagnostico": ["Diabetes Tipo 2", "Hipertensao", "Gripe Forte", "Hipertensa
    "creatinina": [0.8542, 1.2981, 0.7123, 0.9, 1.0, 1.1, 0.8, 1.2981]
})

# Isolando informações (Os colchetes funcionam sempre. O ponto falha se tiver es
```

```
print("--- ISOLANDO UMA SERIES (COLUNA) ---")
display(dados['idade'])
```

Bibliotecas importadas com sucesso!

--- ISOLANDO UMA SERIES (COLUNA) ---

	idade
0	45
1	62
2	38
3	55
4	29
5	71
6	48
7	62

dtype: object

3. Acessando e Compreendendo a Estrutura (Raio-X)

Você não quer imprimir 10 mil linhas de uma vez. Para "espiar os prontuários", usamos:

- `.head()` : 5 primeiras linhas.
- `.tail()` : 5 últimas linhas.
- `.sample()` : Sorteia 1 paciente.

Para entender o tamanho e formato da pesquisa clínica:

- `.shape` : Mostra (linhas, colunas) (Ex: 5000, 15).
- `.columns` : Mostra o nome de todas as colunas.
- `.info()` : Diz os tipos de dados (texto, número) e se há dados faltantes.
- `.describe()` : Calcula automaticamente a média, mínimo e máximo de todos os números.

```
In [32]: print("--- ESPIANDO OS DADOS ---")
display(dados.head()) # 5 primeiras linhas
display(dados.tail()) # 5 últimas linhas
display(dados.sample()) # Sorteia 1 paciente

print("\n--- FORMATO E ESTATÍSTICAS RÁPIDAS ---")
print("Colunas:", dados.columns)
print("Shape:", dados.shape) # Mostra (linhas, colunas)

print("\nInfo:")
dados.info()
```

```
print("\nDescribe:")
display(dados.describe())
```

--- ESPIANDO OS DADOS ---

	paciente	idade	glicemia	sangue	cidade	sexo	data_admissao	diagnostico	ci
0	Ana Lima	45	99.4582	A+	Campina Grande	F	2024-01-15	Diabetes Tipo 2	
1	João	62	140.1234	O+	João Pessoa	Masc	2024-02-10	Hipertensao	
2	Maria	38	NaN	B+	Recife	Feminino	2024-01-20	Gripe Forte	
3	Carlos	55	115.0000	AB+	Natal	M	2024-02-10	Hipertensao	
4	Fernanda	29	-1.0000	O-	Fortaleza	F	2024-03-01	Avaliacao	

	paciente	idade	glicemia	sangue	cidade	sexo	data_admissao	diagnostico	ci
3	Carlos	55	115.0000	AB+	Natal	M	2024-02-10	Hipertensao	
4	Fernanda	29	-1.0000	O-	Fortaleza	F	2024-03-01	Avaliacao	
5	Lucas	71	158.0000	A-	Recife	M	2024-01-05	Diabetes Tipo 1	
6	Beatriz	48	105.0000	B-	Campina Grande	Feminino	2024-02-28	Enxaqueca	
7	João	62	140.1234	O+	João Pessoa	Masc	2024-02-10	Hipertensao	

	paciente	idade	glicemia	sangue	cidade	sexo	data_admissao	diagnostico	creatinina
1	João	62	140.1234	O+	João Pessoa	Masc	2024-02-10	Hipertensao	1.29

```

--- FORMATO E ESTATÍSTICAS RÁPIDAS ---
Colunas: Index(['paciente', 'idade', 'glicemia', 'sangue', 'cidade', 'sexo',
              'data_admissao', 'diagnostico', 'creatinina'],
              dtype='object')
Shape: (8, 9)

```

```

Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   paciente              8 non-null     object
1   idade                 8 non-null     object
2   glicemia              7 non-null     float64
3   sangue                8 non-null     object
4   cidade                8 non-null     object
5   sexo                  8 non-null     object
6   data_admissao         8 non-null     object
7   diagnostico           8 non-null     object
8   creatinina            8 non-null     float64
dtypes: float64(2), object(7)
memory usage: 708.0+ bytes

```

Describe:

	glicemia	creatinina
count	7.000000	8.000000
mean	108.100714	0.995337
std	52.560627	0.220901
min	-1.000000	0.712300
25%	102.229100	0.840650
50%	115.000000	0.950000
75%	140.123400	1.149525
max	158.000000	1.298100

4. Valores Únicos, Repetições e o Pesadelo dos Dados Ausentes

- **Valores Únicos:** `.unique()` lista os valores distintos (ex: Quais os tipos sanguíneos cadastrados?), e `.nunique()` conta quantos valores diferentes existem. `.value_counts()` mostra a frequência (ex: quantos pacientes existem de cada cidade na sua base).
- **Repetições:** Pacientes cadastrados em duplicidade enviam qualquer modelo de IA. Usamos `.duplicated()` para marcar quais linhas são repetidas e `.drop_duplicates()` para remover e limpar a tabela.
- **Dados Ausentes (NaN):** Na saúde, é normal faltar preencher algum dado. O Pandas marca esses buracos como `np.nan` (Não é um número), e eles quebram a IA.

Usamos `.isnull().sum()` para contar, `.replace()` para trocar erros de digitação e `.fillna()` para preencher buracos.

```
In [33]: # Valores Únicos
print("Valores únicos de sangue:", dados['sangue'].unique())
print("Quantidade de tipos de sangue:", dados['sangue'].nunique())

print("\nContagem por cidade:")
display(dados['cidade'].value_counts())

# Repetições no sistema
print("\nMarcando linhas repetidas:")
display(dados.duplicated())

# Remove e limpa a tabela
dados = dados.drop_duplicates()

# O pesadelo: dados ausentes
print("\nContando quantos dados estão faltando:")
print(dados.isnull().sum())

# Troca erro de digitação do aparelho (-1.0) por NaN
dados = dados.replace(-1.0, np.nan)

# Preenche o buraco com zero (Conforme slide)
dados = dados.fillna(0)
```

Valores únicos de sangue: ['A+' 'O+' 'B+' 'AB+' 'O-' 'A-' 'B-']

Quantidade de tipos de sangue: 7

Contagem por cidade:

	count
cidade	
Campina Grande	2
João Pessoa	2
Recife	2
Natal	1
Fortaleza	1

dtype: int64

Marcando linhas repetidas:

0

0 False

1 False

2 False

3 False

4 False

5 False

6 False

7 True

dtype: bool

Contando quantos dados estão faltando:

paciente 0

idade 0

glicemia 1

sangue 0

cidade 0

sexo 0

data_admissao 0

diagnostico 0

creatinina 0

dtype: int64

5. Conversão de tipos

- **Conversão de Tipos:** Às vezes, uma idade como 20 é importada como texto. O comando `.astype()` garante a conversão definitiva.

```
In [34]: # Igualando as variáveis para rodar o código exato dos slides
df = dados.copy()

# Ajustando a rota (Converte a coluna para números inteiros definitivamente)
df['idade'] = df['idade'].astype(int)
```

6. Introdução ao NumPy e Arrays Especiais

Afinal, por que importar o NumPy se temos o Pandas? Se o Pandas é o organizador de tabelas, o NumPy é o motor matemático dele. Por trás das cortinas, cada coluna do Pandas guarda seus números dentro de um Array do NumPy para conseguir processar os cálculos tão rápido.

Arrays vs Listas:

- A **Lista** do Python é como uma maleta médica genérica: cabe gaze, seringa, esparadrapo (aceita texto, número, booleanos tudo junto). É flexível, mas muito lenta para a IA calcular coisas.

- O **Array** do NumPy é como uma bandeja cirúrgica: só aceita um tipo de coisa (ex: apenas números). Essa rigidez permite que o computador processe milhares de operações matemáticas de uma vez só (Vetorização).

Arrays Especiais:

- `np.arange()` : Gera uma sequência numérica espaçada, como uma régua.
- `np.zeros()` : Cria matriz preenchida com zeros. Muito usado para inicializar pesos em IA.
- `np.ones()` : Semelhante ao zeros, mas preenche a estrutura com o número 1.

In [35]: *# Criação exata das funções mostradas nos slides de NumPy*

```
# 1. np.arange() - Sequência espaçada
array_regua = np.arange(10) # Gera de 0 a 9
print("np.arange(10):")
print(array_regua)

# 2. np.zeros() - Matriz de zeros
array_zeros = np.zeros(5)
print("\nnp.zeros(5):")
print(array_zeros)

# 3. np.ones() - Matriz de uns
matriz_ones = np.ones(5)
print("\nnp.ones(5):")
print(matriz_ones)
```

```
np.arange(10):
[0 1 2 3 4 5 6 7 8 9]
```

```
np.zeros(5):
[0. 0. 0. 0. 0.]
```

```
np.ones(5):
[1. 1. 1. 1. 1.]
```

Observação: nos slides da aula, as tabelas de "antes" e "depois" foram geradas separadamente (sem `display()`), já que cada imagem foi capturada de forma independente. Aqui no notebook, usamos o `display()` apenas para facilitar os testes, permitindo mostrar as duas tabelas dentro de uma única célula de código. Mas ele não é obrigatório: é apenas uma função de exibição, e o código funciona normalmente sem ela. A diferença é só na forma como o resultado aparece na tela.

7. Formatando/transformando: .replace(), .round(), pd.to_datetime(), .dt.year, .str.strip(), .str.contains()

.str.strip()

Quando os dados são digitados manualmente em um sistema, é muito comum que sobre espaços invisíveis no início ou no final do texto (por exemplo, " Ana Lima " em vez de "Ana Lima"). Isso pode atrapalhar buscas e comparações depois. O método `.str.strip()` remove esses espaços extras.

```
In [36]: import pandas as pd

dados = {
    'paciente': [' Ana Lima ', 'Carlos Souza ', ' Beatriz Santos '],
    'convenio': [' Unimed ', ' SUS', 'Bradesco '],
    'idade': [45, 62, 38]
}
df = pd.DataFrame(dados)

print("ANTES DO STRIP:")
print(df['paciente'].tolist())

# 2. Aplicando a limpeza com .str.strip()
df['paciente'] = df['paciente'].str.strip()
df['convenio'] = df['convenio'].str.strip()

print("\nDEPOIS DO STRIP:")
print(df['paciente'].tolist())
```

ANTES DO STRIP:

```
[' Ana Lima ', 'Carlos Souza ', ' Beatriz Santos ']
```

DEPOIS DO STRIP:

```
['Ana Lima', 'Carlos Souza', 'Beatriz Santos']
```

.replace()

Em bases clínicas reais, é comum encontrar a mesma informação escrita de formas diferentes ('F', 'Fem', 'Feminino'). O método `.replace()` permite padronizar esses valores, trocando cada versão pela forma correta.

```
In [37]: dados = {
    'paciente': ['Ana Lima', 'Carlos Souza', 'Beatriz Santos', 'Lucas Rocha'],
    'sexo': ['F', 'Masc', 'Feminino', 'M']
}
df = pd.DataFrame(dados)

# Antes da padronização
print("Antes:")
display(df[['paciente', 'sexo']])

# Aplicando .replace() para padronizar
df['sexo'] = df['sexo'].replace({'F': 'Feminino', 'M': 'Masculino', 'Masc': 'Mas'})
print("\nDepois:")
display(df[['paciente', 'sexo']])
```

Antes:

	paciente	sexo
0	Ana Lima	F
1	Carlos Souza	Masc
2	Beatriz Santos	Feminino
3	Lucas Rocha	M

Depois:

	paciente	sexo
0	Ana Lima	Feminino
1	Carlos Souza	Masculino
2	Beatriz Santos	Feminino
3	Lucas Rocha	Masculino

.round()

Exames laboratoriais costumam vir com muitas casas decimais, o que dificulta a leitura. O método `.round()` ajusta o número de casas decimais de uma coluna numérica.

```
In [38]: dados = {
    'paciente': ['Ana Lima', 'Carlos Souza', 'Beatriz Santos'],
    'glicemia': [99.4582, 140.1234, 88.7891],
    'creatinina': [0.8542, 1.2981, 0.7123]
}
df = pd.DataFrame(dados)

print("Antes")
display(df[['paciente', 'glicemia', 'creatinina']])

# Aplicando o .round() para 1 casa decimal na glicemia e 2 na creatinina
df['glicemia'] = df['glicemia'].round(1)
df['creatinina'] = df['creatinina'].round(2)

print("Depois")
display(df[['paciente', 'glicemia', 'creatinina']])
```

Antes

	paciente	glicemia	creatinina
0	Ana Lima	99.4582	0.8542
1	Carlos Souza	140.1234	1.2981
2	Beatriz Santos	88.7891	0.7123

Depois

	paciente	glicemia	creatinina
0	Ana Lima	99.5	0.85
1	Carlos Souza	140.1	1.30
2	Beatriz Santos	88.8	0.71

pd.to_datetime()

Quando lemos um arquivo (como um CSV), o pandas geralmente interpreta datas como texto simples (tipo object). Isso significa que não conseguimos fazer operações como "quantos dias se passaram" ou "filtrar só os atendimentos de 2024". A função `pd.to_datetime()` converte esse texto em um formato de data de verdade (datetime64), liberando esses recursos.

Nota: Não é possível fazer operações de data e hora enquanto a coluna estiver como texto (`str`). É por isso que essa conversão é sempre um dos primeiros passos ao tratar dados com datas.

```
In [39]: # Base com data de admissão como texto (string)
dados = {
    'paciente': ['Ana Lima', 'Carlos Souza', 'Beatriz Santos'],
    'data_admissao': ['2023-03-15', '2024-07-20', '2024-09-02']
}
df = pd.DataFrame(dados)

print(df[['data_admissao']].dtypes)

print("Antes:")
display(df[['paciente', 'data_admissao']])

# 2. Convertendo para data real e extraindo apenas o ano
df['data_admissao'] = pd.to_datetime(df['data_admissao'])
df['ano_admissao'] = df['data_admissao'].dt.year

print(df[['data_admissao']].dtypes)
print("Depois:")
display(df[['paciente', 'data_admissao']])
```

```
data_admissao    object
dtype: object
Antes:
```

	paciente	data_admissao
0	Ana Lima	2023-03-15
1	Carlos Souza	2024-07-20
2	Beatriz Santos	2024-09-02

```
data_admissao    datetime64[ns]
dtype: object
Depois:
```

	paciente	data_admissao
0	Ana Lima	2023-03-15
1	Carlos Souza	2024-07-20
2	Beatriz Santos	2024-09-02

.dt.year

Depois que uma coluna está no formato de data, podemos usar o acessor `.dt` para extrair partes específicas dela, como o ano, o mês ou o dia.

```
In [40]: df['ano_admissao'] = df['data_admissao'].dt.year
df[['paciente', 'data_admissao', 'ano_admissao']]
```

```
Out[40]:
```

	paciente	data_admissao	ano_admissao
0	Ana Lima	2023-03-15	2023
1	Carlos Souza	2024-07-20	2024
2	Beatriz Santos	2024-09-02	2024

Dica: o mesmo acessor funciona para outras partes da data, como `.dt.month` (mês) e `.dt.day` (dia).

.str.contains()

Muitas vezes precisamos filtrar linhas que contenham uma palavra específica dentro de um texto — por exemplo, encontrar todos os pacientes cujo diagnóstico menciona "diabetes", mesmo que existam variações de maiúsculas e minúsculas. O parâmetro `case=False` garante que a busca ignore essa diferença.

```
In [41]: dados = {
    'paciente': ['Ana Lima', 'Carlos Souza', 'Beatriz Santos', 'Lucas Rocha'],
    'diagnostico': ['Diabetes tipo 2', 'Hipertensão Arterial', 'Pré-diabetes', '']
}
df = pd.DataFrame(dados)

print("Base Original:")
display(df[['paciente', 'diagnostico']])

# Filtrando pacientes com qualquer menção a 'diabetes' (case=False ignora maiúsc
filtro_diabetes = df['diagnostico'].str.contains('diabetes', case=False)
df_diabetes = df[filtro_diabetes]

# Apenas pacientes com diabetes
print("Pacientes Filtrados (com Diabetes):")
display(df_diabetes[['paciente', 'diagnostico']])
```

Base Original:

	paciente	diagnostico
0	Ana Lima	Diabetes tipo 2
1	Carlos Souza	Hipertensão Arterial
2	Beatriz Santos	Pré-diabetes
3	Lucas Rocha	Gripe Comum

Pacientes Filtrados (com Diabetes):

	paciente	diagnostico
0	Ana Lima	Diabetes tipo 2
2	Beatriz Santos	Pré-diabetes

8. Ordenação: .sort_values()

.sort_values()

Esse método reorganiza as linhas do DataFrame de acordo com os valores de uma coluna. Na prática, isso é muito útil para priorizar atendimentos. Por exemplo, listando primeiro os pacientes com maior glicemia (maior risco).

```
In [42]: dados = {
    'paciente': ['Ana Lima', 'Carlos Souza', 'Beatriz Santos', 'Lucas Rocha'],
    'idade': [45, 62, 38, 71],
    'glicemia': [99.5, 140.1, 88.8, 185.0]
}
df = pd.DataFrame(dados)

print("Antes:")
display(df[['paciente', 'idade', 'glicemia']])

# Ordenando da maior para a menor glicemia (decrescente)
df_ordenado = df.sort_values(by='glicemia', ascending=False)

print("Depois:")
display(df_ordenado[['paciente', 'idade', 'glicemia']])
```

Antes:

	paciente	idade	glicemia
0	Ana Lima	45	99.5
1	Carlos Souza	62	140.1
2	Beatriz Santos	38	88.8
3	Lucas Rocha	71	185.0

Depois:

	paciente	idade	glicemia
3	Lucas Rocha	71	185.0
1	Carlos Souza	62	140.1
0	Ana Lima	45	99.5
2	Beatriz Santos	38	88.8

Nota: Para ordenar do menor para o maior, basta usar `ascending=True` (que já é o padrão, então também funciona sem especificar nada).

9. Índice do DataFrame: `.set_index()`, `.reset_index()` (incluindo indexar por data como base pra séries temporais)

`.set_index()` e `.reset_index()`

Todo DataFrame tem um índice: os números (0, 1, 2...) que aparecem à esquerda da tabela. Podemos trocar esse índice numérico por uma coluna que identifique melhor cada linha, como o ID do paciente.

```
In [43]: dados = {
    'id_paciente': [101, 102, 103],
    'nome': ['Ana Lima', 'Carlos Souza', 'Beatriz Santos'],
    'setor': ['UTI', 'Triagem', 'Enfermaria']
}
df = pd.DataFrame(dados)

# Definindo o ID como índice
df_idx = df.set_index('id_paciente')
print("Com id_paciente no índice (.set_index):")
display(df_idx)

# Resetando para o índice numérico padrão
df_reset = df_idx.reset_index()
print("Índice restaurado (.reset_index):")
display(df_reset)
```

Com id_paciente no índice (.set_index):

	nome	setor
id_paciente		
101	Ana Lima	UTI
102	Carlos Souza	Triagem
103	Beatriz Santos	Enfermaria

Índice restaurado (.reset_index):

	id_paciente	nome	setor
0	101	Ana Lima	UTI
1	102	Carlos Souza	Triagem
2	103	Beatriz Santos	Enfermaria

Indexação por data

Quando trabalhamos com dados que se repetem ao longo do tempo (como o número de internações por dia), é muito útil transformar a data em índice. Isso permite consultar um dia específico diretamente com `.loc[]`.

```
In [44]: dados_tempo = {
          'data': pd.to_datetime(['2024-05-01', '2024-05-02', '2024-05-03', '2024-05-04']),
          'internacoes_diarias': [12, 18, 15, 22]
        }
df_tempo = pd.DataFrame(dados_tempo).set_index('data')

print("Série Temporal (Índice = DatetimeIndex):")
display(df_tempo)
```

Série Temporal (Índice = DatetimeIndex):

internacoes_diarias	
data	
2024-05-01	12
2024-05-02	18
2024-05-03	15
2024-05-04	22

```
In [45]: print("Internações em 02/05/2024:")
df_tempo.loc['2024-05-02']
```

Internações em 02/05/2024:

```
Out[45]:
```

2024-05-02	
internacoes_diarias	18

dtype: int64

10. Agrupamento e agregação: `.groupby()`

`.groupby()`

O `.groupby()` agrupa os dados de acordo com uma coluna de categorias (por exemplo, o diagnóstico) e permite calcular resumos estatísticos para cada grupo, como a média, a soma ou a contagem.

```
In [46]: dados = {
    'paciente': ['Ana', 'Carlos', 'Beatriz', 'Lucas', 'Marina', 'João'],
    'diagnostico': [
        'Diabetes', 'Hipertensão', 'Diabetes',
        'Gripe', 'Hipertensão', 'Diabetes'
    ],
    'idade': [62, 55, 48, 23, 67, 71]
}

df = pd.DataFrame(dados)

print("Base de Pacientes:")
df[['paciente', 'diagnostico', 'idade']]

# Média de idade por diagnóstico arredondada
resumo = df.groupby('diagnostico')[['idade']].mean().round(1)

print("Média de Idade por Diagnóstico:")
resumo.reset_index()
```

Base de Pacientes:

Média de Idade por Diagnóstico:

```
Out[46]:
```

	diagnostico	idade
0	Diabetes	60.3
1	Gripe	23.0
2	Hipertensão	61.0

Nota: Outras agregações úteis: `.sum()` (soma), `.count()` (contagem de registros) e `.max()` / `.min()` (maior e menor valor).

11. Criando novas variáveis: coluna condicional com `.loc[]`, coluna calculada com `.assign()`

`.loc[]`

Podemos criar uma nova coluna cujo valor depende de uma regra lógica. Um exemplo clássico na saúde é classificar a glicemia como "Normal" ou "Alterada" de acordo com um valor de referência clínico (neste caso, 126 mg/dL, o limiar usado para diagnóstico de diabetes em jejum).

```
In [47]: dados = {
    'paciente': ['Ana Lima', 'Carlos Souza', 'Beatriz Santos', 'Lucas Rocha'],
```

```

    'glicemia': [95.0, 145.0, 88.0, 180.0]
}
df = pd.DataFrame(dados)

print("Base Original:")
display(df[['paciente', 'glicemia']])

# Criando coluna condicional com .loc[]
df['status'] = 'Normal'
df.loc[df['glicemia'] >= 126.0, 'status'] = 'Alterada'

print("Com Coluna Condicional:")
display(df[['paciente', 'glicemia', 'status']])

```

Base Original:

	paciente	glicemia
0	Ana Lima	95.0
1	Carlos Souza	145.0
2	Beatriz Santos	88.0
3	Lucas Rocha	180.0

Com Coluna Condicional:

	paciente	glicemia	status
0	Ana Lima	95.0	Normal
1	Carlos Souza	145.0	Alterada
2	Beatriz Santos	88.0	Normal
3	Lucas Rocha	180.0	Alterada

.assign()

O `.assign()` permite criar novas colunas a partir de uma função aplicada sobre colunas já existentes, de forma organizada e sem precisar reescrever o DataFrame inteiro. Um exemplo prático é calcular o IMC a partir do peso e da altura.

```

In [48]: dados_imc = {
    'paciente': ['Ana Lima', 'Carlos Souza', 'Beatriz Santos'],
    'peso': [68.0, 85.0, 54.0],
    'altura': [1.65, 1.75, 1.60]
}
df_imc = pd.DataFrame(dados_imc)

print("Antes do Cálculo:")
display(df_imc[['paciente', 'peso', 'altura']])

# Definindo a função clínica para cálculo do IMC
def calcular_imc(tabela):
    return (tabela['peso'] / (tabela['altura'] ** 2)).round(1)

# Aplicando com .assign()

```

```
df_imc = df_imc.assign(imc=calcular_imc)

print("Depois (.assign):")
display(df_imc[['paciente', 'peso', 'altura', 'imc']])
```

Antes do Cálculo:

	paciente	peso	altura
0	Ana Lima	68.0	1.65
1	Carlos Souza	85.0	1.75
2	Beatriz Santos	54.0	1.60

Depois (.assign):

	paciente	peso	altura	imc
0	Ana Lima	68.0	1.65	25.0
1	Carlos Souza	85.0	1.75	27.8
2	Beatriz Santos	54.0	1.60	21.1

12. Removendo colunas: dados.drop('COLUNA', axis=1)

.drop()

Às vezes precisamos eliminar colunas desnecessárias ou que contenham informações sensíveis antes de compartilhar uma base de dados. Usamos o parâmetro `axis=1` para indicar que queremos remover uma coluna (e não uma linha).

```
In [49]: dados = {
    'paciente': ['Ana Lima', 'Carlos Souza', 'Beatriz Santos'],
    'diagnostico': ['Diabetes', 'Hipertensão', 'Diabetes'],
    'observacoes_internas': ['Anotação 1', 'Anotação 2', 'Anotação 3']
}
df = pd.DataFrame(dados)

print("Base Original (com observacoes_internas):")
display(df[['paciente', 'diagnostico', 'observacoes_internas']])

# Removendo a coluna desnecessária
df_limpo = df.drop('observacoes_internas', axis=1)

print("Base Limpa (sem a coluna):")
display(df_limpo)
```

Base Original (com observacoes_internas):

	paciente	diagnostico	observacoes_internas
0	Ana Lima	Diabetes	Anotação 1
1	Carlos Souza	Hipertensão	Anotação 2
2	Beatriz Santos	Diabetes	Anotação 3

Base Limpa (sem a coluna):

	paciente	diagnostico
0	Ana Lima	Diabetes
1	Carlos Souza	Hipertensão
2	Beatriz Santos	Diabetes

13. Salvando resultados: .to_csv(), .to_excel()

.to_csv() e .to_excel()

Depois de tratar os dados, o passo final costuma ser exportar o resultado para um arquivo que possa ser compartilhado ou usado em relatórios.

```
In [50]: dados_finais = {
    'paciente': ['Ana Lima', 'Carlos Souza', 'Beatriz Santos'],
    'diagnostico': ['Diabetes tipo 2', 'Hipertensão Arterial', 'Pré-diabetes'],
    'glicemia': [99.5, 140.1, 88.8]
}
df_final = pd.DataFrame(dados_finais)

# Salvando em CSV
df_final.to_csv('relatorio_pacientes.csv', index=False, encoding='utf-8-sig')

# Salvando em Excel (.xlsx)
df_final.to_excel('relatorio_pacientes.xlsx', index=False, sheet_name='Triagem')
```

```
In [52]: !jupyter nbconvert --to html /content/Pandas_para_saúde.ipynb
```

```
[NbConvertApp] Converting notebook /content/Pandas_para_saúde.ipynb to html
[NbConvertApp] Writing 480762 bytes to /content/Pandas_para_saúde.html
```